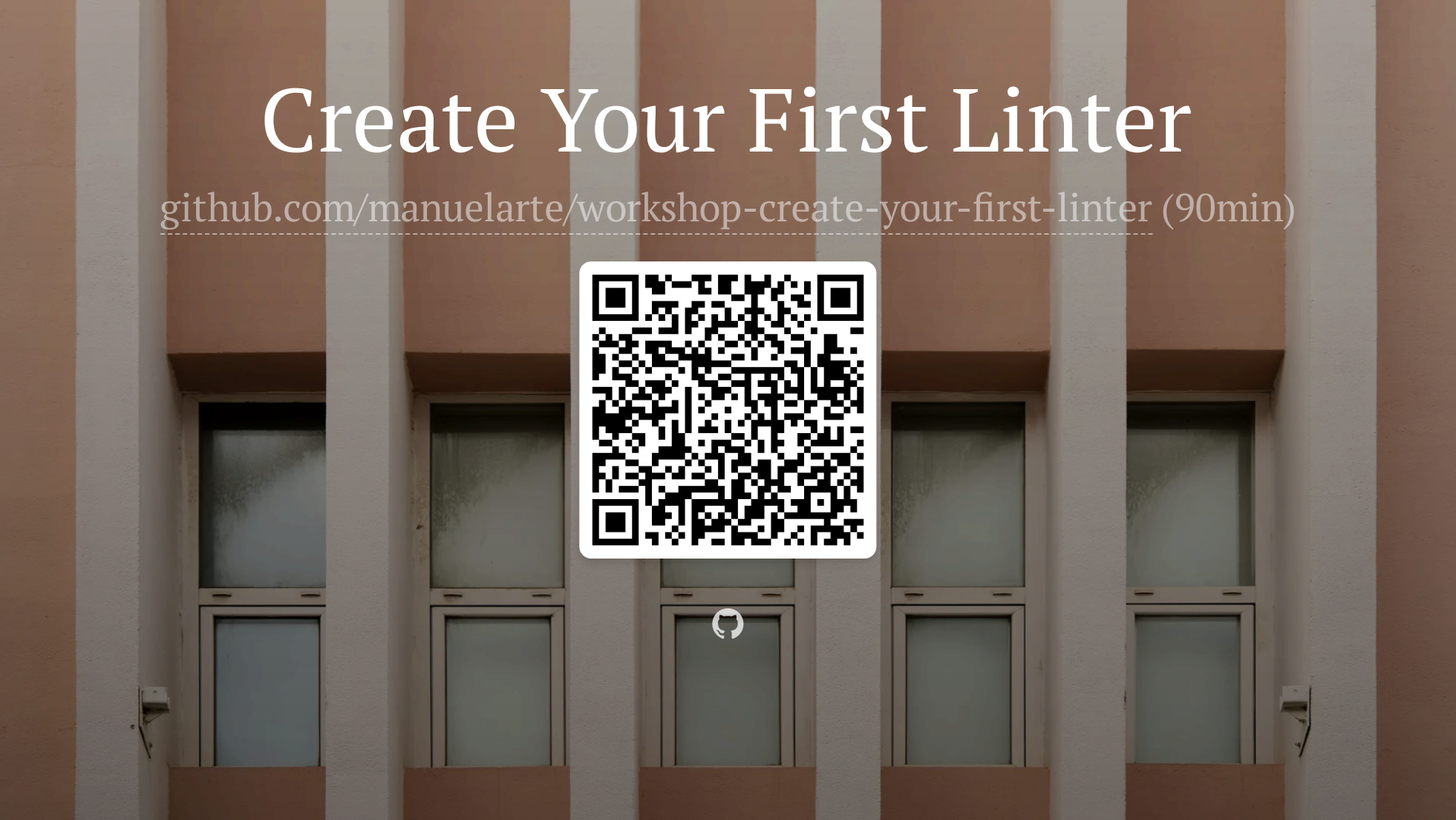
Create Your First Linter

github.com/manuelarte/workshop-create-your-first-linter (90min)



What is a linter?

Automatically analyze source code for potential errors, stylistic issues, and violations of coding conventions

About Me

me.go

```
var Me = Developer{
  Name: "Manuel Doncel Martos",
  Skills: [][]string {
        {"☕Java", "Spring Boot"},
        {"🦫Go", "🐍Python"},
        {"Kubernetes", "Docker"},
    },
    Interests: []string {
        "Open Source",
        "Domain Driven Design",
        "⚽Football",
    },
}
```

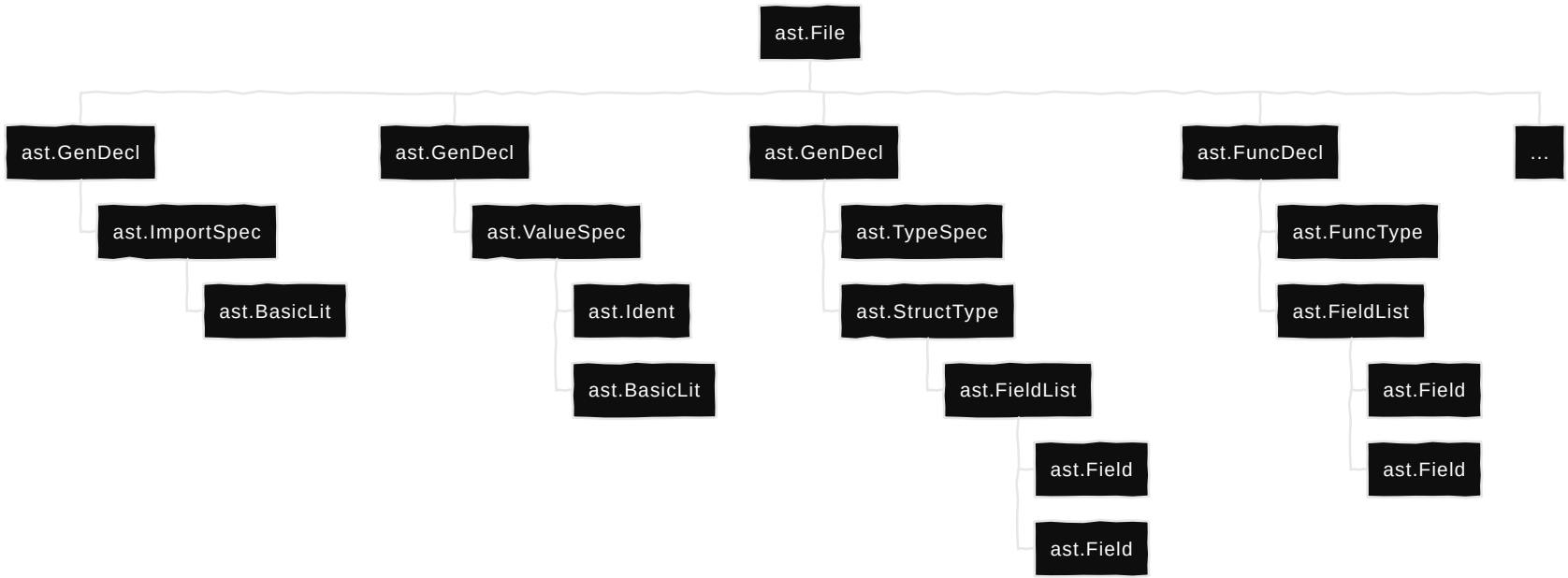
Table of contents

1. Abstract Syntax Tree (AST)
2. Tools Analyzer
3. My First Linter
    1. Definition
    2. Implementation
4. Run Linter
    1. Standalone
    2. Golangci-lint Plugin
    3. Suggest a fix

Abstract Syntax Tree (AST)

Tree that represents the structure of a program or code snippet

AST Example



File to test

Tools Analyzer

Package golang.org/x/tools

- Node: represents a node in the AST.
- Analyzer: (a.k.a. linter) contains the business logic, analyzes the AST and reports issues.
- Pass: represents a single run of an Analyzer on a set of files.
- Diagnostic: represents an issue found by an Analyzer.

AST Exercise

AST Exercise

- Clone/Fork the repository.
- Run tests of astexample.
- Analyze the output of `ast.Print(pass.Fset, file)`.
- Check the AST of different Go specs:
  - Const.
  - Var.
  - etc.

My First Linter

Definition

Prefix unexported globals with '_'

Prefix unexported top-level vars and consts with _ to make it clear when they are used that they are global symbols.

❌ Bad

```
const myConstant = "myConstant"
const errNotFound = "not found"
```

✅ Good

```
const _myConstant = "myConstant"
const errNotFound = "not found"
```

Implementation

- Write test cases first. Using analysistest.
- Implement business logic.

Define test cases

- Write a file with the different cases to test.
- Add inlined comments where needed with the expected output.

Let’s

Comments

- Stylistic linters are very useful in big projects or in big teams.
- Be consistent.

Run Linter

Standalone

```
go build -o unexportedconstantscheck
cd myproject
unexportedconstantscheck --fix ./...
```

Golangci-lint Plugin

Golangci-lint is a fast linters runner for Go.

- Widely used.
- More than 100 linters.
- Linters & Formatters.

Golangci-lint Plugin

- Custom golangci-lint binary.
- Custom linter is added to that binary.
- Add plugin library and register the linter in `init()` function.
- Run `make lint` and check that the linter is executed.

Let’s

Suggest a fix

- Modify the source code to fix the issue.
- Run the linter and check the issue is fixed, but…

```
analysis.Diagnostic{
    ...
    SuggestedFixes: []analysis.SuggestedFix{
        {
            Message: msg,
            TextEdits: []analysis.TextEdit{
                {
                    Pos: i.Pos(),
                    End: i.End(),
                    NewText: "", // TODO
                },
            },
```

Q/A

